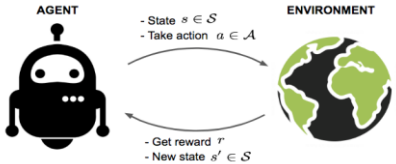
Robustesse des algorithmes d'Apprentissage par Renforcement
Étude de la stabilisation de pendules inversés via la théorie du contrôle

Stabilisation d'un Pendule Inversé
Commande LQR vs Apprentissage par Renforcement PPO

Équipe de Recherche

Gevorg Andreasyan Hicham Habchane Khadijetou Soumare Mouhamed Ndiaye

Encadrant : Thibault Liard



Année universitaire : 2025-2026

Plan de la Présentation

01	**Modélisation**	*Système physique, équations, linéarisation, stabilité, contrôlabilité*
02	**Commande LQR**	*Retour d'état, critère J, Riccati, gain K, simulations*
03	**Apprentissage par Renforcement (PPO)**	*Principe RL, architecture PPO, environnement, résultats*
04	**Comparaison LQR vs PPO**	*Performances, domaine de stabilité.*
05		

Introduction — Problématique et Objectifs

« Comment un système physique peut-il apprendre à se stabiliser par lui-même, sans qu'on lui donne explicitement les règles mathématiques qui le régissent ? »

Contexte

- Drones, robots, véhicules autonomes
- Limites des méthodes classiques
- Essor de l'IA et du RL

Problématique

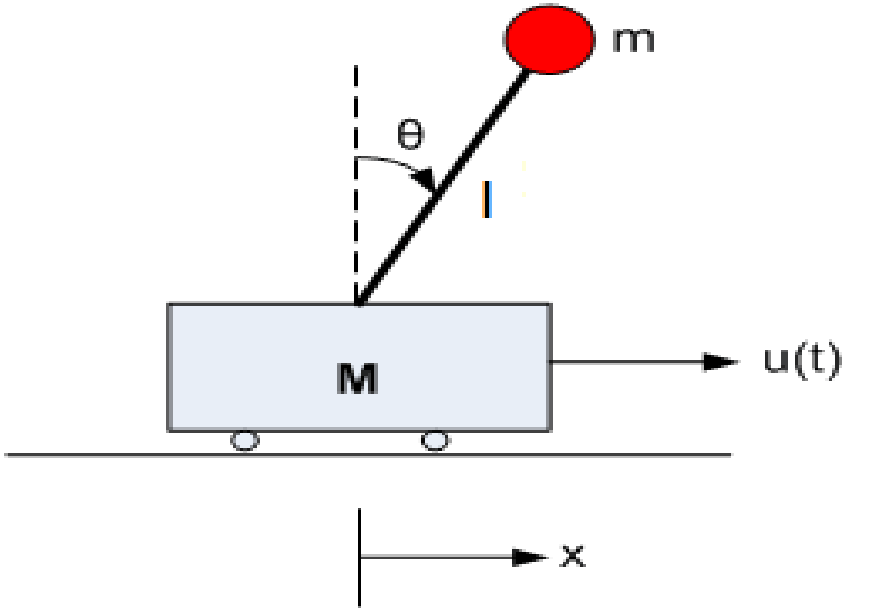
- Comment stabiliser un pendule inversé ?
- PPO peut-il surpasser le LQR ?
- Comparaison équitable ($U_{max} = 4$ N)

Objectifs

- Modéliser le système physique
- Concevoir la commande LQR
- Implémenter l'agent PPO
- Comparer les deux approches

Méthodologie : Analyse mathématique · Implémentation Python · Validation par simulation

1. Modélisation — Le Système Physique



Variables d'état

x	Position du chariot [m]
$\dot{x}$	Vitesse du chariot [m/s]
θ	Angle / verticale [rad]
$\dot{\theta}$	Vitesse angulaire [rad/s]
$u(t)$	Force de commande [N]

$M = 1\ kg \cdot m = 0.1\ kg \cdot l = 0.5\ m \cdot g = 9.81\ m/s^2$

Objectif : maintenir $\theta \to 0$ (pendule vertical) et $x \to 0$ (chariot centré) via $u(t)$.

1. Modélisation — Équations du Mouvement et Linéarisation

Équations non-linéaires exactes

A

B

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = u \\ l\ddot{\theta} + \ddot{x}\cos\theta = g\sin\theta \end{cases}$$

Non-linéaires : termes sinϑ, cosϑ et ϑ²

Système linéarisé

$$(M+m)\ddot{x} + ml\ddot{\theta} = u$$
$$l\ddot{\theta} + \ddot{x} = g\theta$$

Valide pour $\vartheta \lesssim 15°$

Points d'équilibre (u = 0, θ = 0, ẍ = 0) → g sinθ = 0 :

θ = π (vers le bas)
Équilibre STABLE ✓

θ = 0 (vers le haut)
Équilibre INSTABLE ✗

Forme d'état : $\dot{X} = AX + Bu$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -mg/M & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & (M+m)g/Ml & 0 \end{bmatrix} \qquad B = \begin{bmatrix} 0 \\ 1/M \\ 0 \\ -1/Ml \end{bmatrix}$$

$X = [x, \dot{x}, \theta, \dot{\theta}]^T$

Notre objectif : stabiliser $\vartheta = 0$

1. Modélisation — Analyse Spectrale du Système Non Contrôlé

Sans commande (u=0) : $\dot{X} = AX \rightarrow X(t) = e^{At} X_0 \rightarrow$ comportement dicté par les valeurs propres de A

Règle fondamentale

Chaque mode évolue comme $e\lambda^t$:

$\lambda < 0$	$\lambda > 0$
$e\lambda^t \rightarrow 0$ ✓ Stable	$e\lambda^t \rightarrow \infty$ ✗ Instable

$\Longrightarrow$ **Un seul $\lambda > 0$ suffit à rendre le système instable**

$\Longrightarrow$ Il faut une commande pour modifier les λ de A−BK

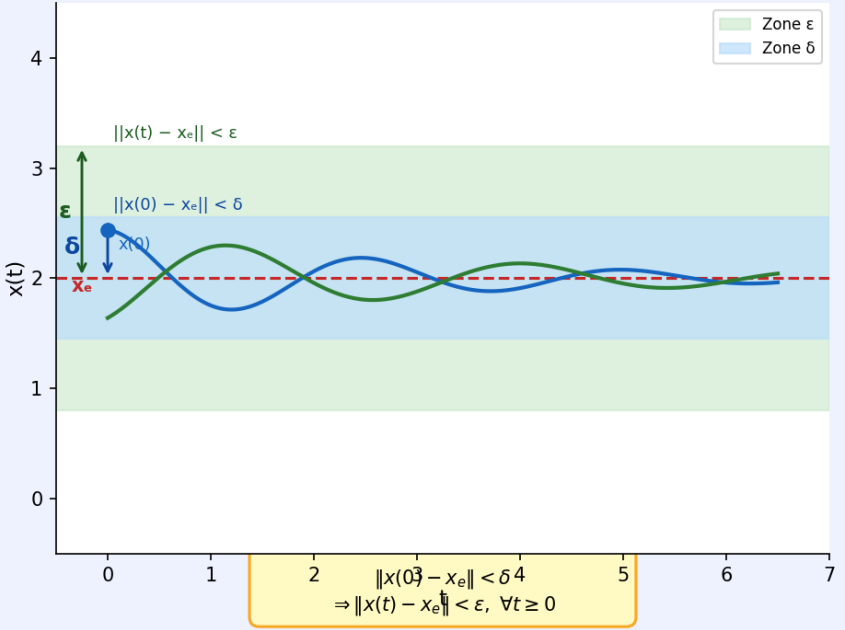
Valeurs propres de A (M=1, m=0.1, l=0.5)

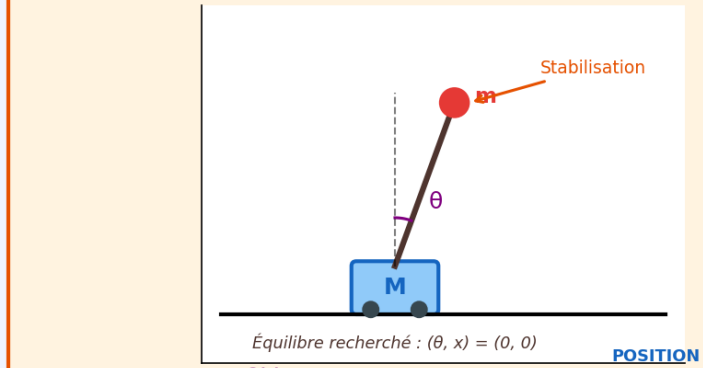
$\lambda_{1,2} = 0$ (double)	Mode neutre — chariot libre
$\lambda_3 = -4.646$	Mode stable ✓
$\lambda_4 = +4.646$	INSTABLE ✗ → $e\lambda^t \rightarrow \infty$

$\lambda_4 = +4.646 > 0$: le pendule s'éloigne exponentiellement de la verticale
→ une commande est INDISPENSABLE !

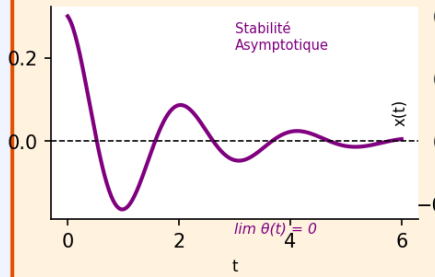
1. Modélisation — Définition de la Stabilité au Sens de Lyapunov

1. DÉFINI (AVEC COND **STABILITÉ AU SENS DE LYAPUNOV : DÉFINITIONS ET EXEMPLES** **ICATION : LE INVERSÉ**

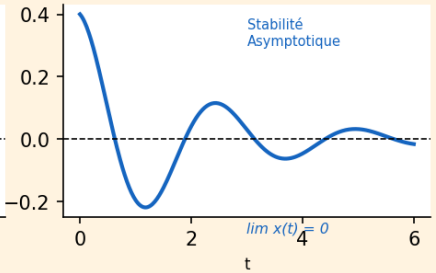




ANGLE θ(t)



POSITION DU CHARIOT x(t)



Équilibre recherché : (θ, x) = (0, 0)
Le contrôleur maintient ||x(t) − xₑ|| < ε pour ||x(0) − xₑ|| < δ

1. Modélisation — Contrôlabilité du Système

Définition :

Un système est dit contrôlable si, pour tout état initial x_0 et tout état final x_1, on peut trouver une commande u(t) qui amène le système de x_0 à x_1 en un temps fini T.

Critère de Kalman

Dans le cas linéaire — comme notre système —
la contrôlabilité se vérifie avec le critère de Kalman :

$$\mathcal{C} = [B \quad AB \quad A^2B \quad A^3B] \qquad \text{rang}(\mathcal{C}) = n \iff \text{Contrôlable}$$

Application numérique

rang($\mathcal{C}$) = 4 = n ✓ Système contrôlable

u agit simultanément sur :
x, ẋ, θ et θ̇

Interprétation :

Un rang plein (rang = n) signifie que u peut agir sur toutes les directions de l'espace d'état.
Quel que soit x_0, on peut atteindre n'importe quel état final x_1 en un temps fini.
La contrôlabilité garantit l'existence d'une commande u(t) ramenant le système à l'équilibre $x_1 = 0$.

2. Commande LQR — Principe du Retour d'État

$$u(t) = -Kx(t) = -k_1 \cdot x - k_2 \cdot \dot{x} - k_3 \cdot \theta - k_4 \cdot \dot{\theta}$$

$k_1 \cdot x$	$k_2 \cdot \dot{x}$	$k_3 \cdot \theta$	$k_4 \cdot \dot{\theta}$
Position chariot	**Vitesse chariot**	**Angle pendule**	**Vit. angulaire**
Rappelle vers x = 0	Amortit les mouvements	LE PLUS IMPORTANT Redresse le pendule	Anticipe les oscillations

Calcul du gain optimal K :

```
K = R⁻¹ · Bᵀ · P   [Python : solve_continuous_are]
```

$$K = [-3.16 \quad -4.25 \quad -38.75 \quad -8.22]$$

Gains négatifs → u corrige dans la direction opposée à l'écart.

2. Commande LQR — Fonction de Coût et Équation de Riccati

Fonction de coût J à minimiser

$$J = \int_0^{\infty} (X^T Q X + u R u)\, dt$$

`Q = diag(1, 0, 10, 0)`	Pénalise x (×1) et θ (×10) → priorité : redresser le pendule
`R = 0.1`	Pénalise l'effort u → force modérée
Compromis Q/R	Q grande → rapide | R grand → économe Réglage selon les besoins de l'application

ϑ pénalisé 10× plus que x → priorité : redresser le pendule avant de centrer le chariot.

Équation de Riccati

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

puis $K = R^{-1} \cdot B^T \cdot P$

P = solve_continuous_are(A,B,Q,R)

Valeurs propres de A−BK :

$\lambda_{1,2} = -5.04 \pm 1.99i$
$\lambda_{3,4} = -1.06 \pm 1.00i$
Toutes Re(λ) < 0 ✓ Stabilité garantie

La résolution de Riccati garantit $Re(\lambda_i) < 0$ pour toutes les valeurs propres de A−BK.

Riccati garantit que TOUTES les valeurs propres de A−BK ont $Re(\lambda_i) < 0$ → Système asymptotiquement stable

2. Commande LQR — Pourquoi la Commande Stabilise ?

Boucle fermée : $\dot{x}(t) = (A-BK)x(t) \rightarrow x(t) = e^{(A-BK)t} x_0$

$\mathrm{Re}(\lambda_i) < 0$

$e^{\lambda_i t} \rightarrow 0$ quand $t \rightarrow \infty$

Le mode est STABLE ✓
Le système revient à l'équilibre

$\mathrm{Re}(\lambda_i) > 0$

$e^{\lambda_i t} \rightarrow \infty$ quand $t \rightarrow \infty$

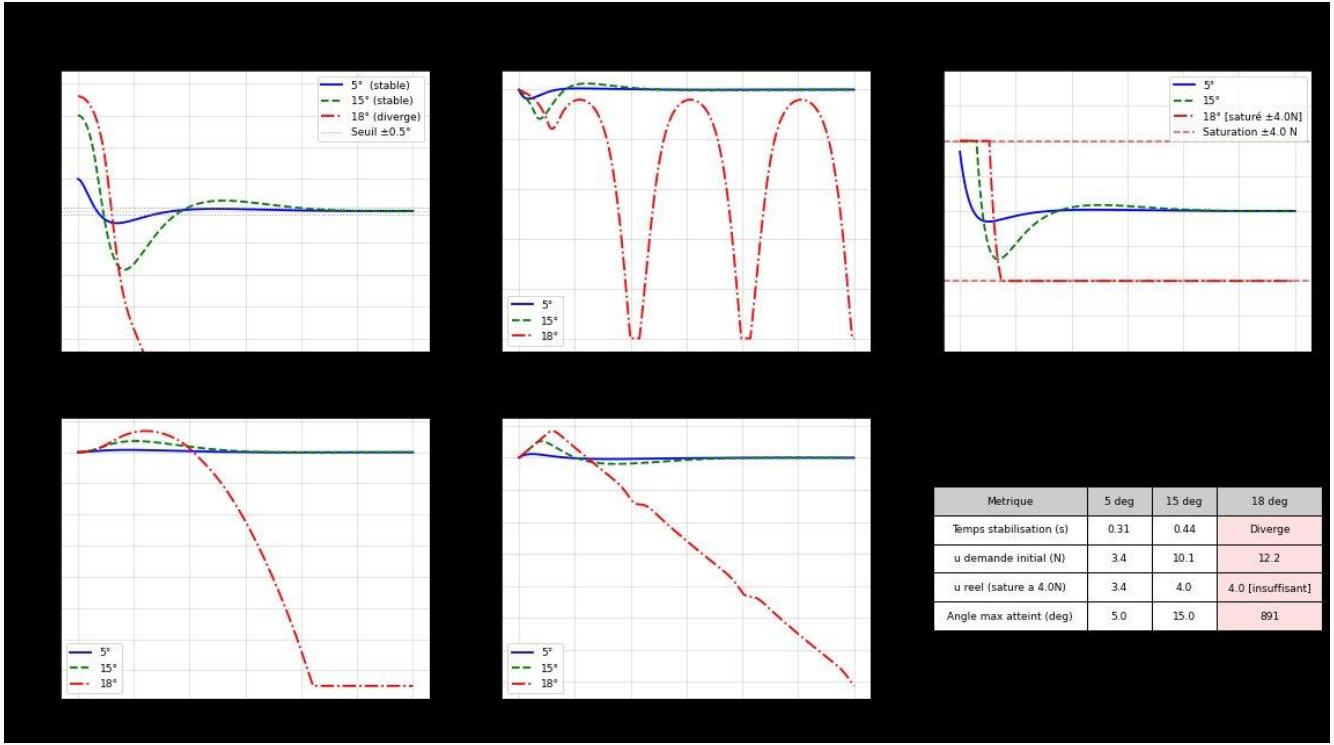
Le mode est INSTABLE ✗
Le système diverge

Propriété fondamentale du LQR :

La résolution de l'équation de Riccati garantit que TOUTES les valeurs propres de $A-BK$ ont $\mathrm{Re}(\lambda_i) < 0$.
Ainsi chaque terme $e^{\lambda_i t} \rightarrow 0$, donc $x(t) \rightarrow 0 \implies \theta(t) \rightarrow 0$ et $x(t) \rightarrow 0$: stabilité asymptotique.

$\lambda_{1,2} = -5.04 \pm 1.99i$ $\lambda_{3,4} = -1.06 \pm 1.00i$ → Toutes $\mathrm{Re}(\lambda) < 0$ ✓ Système asymptotiquement stable

2. Résultats LQR — Modèle Non-Linéaire + Saturation (U_max = 4 N)



Metrique	5 deg	15 deg	18 deg
Temps stabilisation (s)	0.31	0.44	Diverge
u demande initial (N)	3.4	10.1	12.2
u reel (sature a 4.0N)	3.4	4.0	4.0 [insuffisant]
Angle max atteint (deg)	5.0	15.0	891

t_stab (s)	0.31	0.44	Diverge
u demandé	3.4 N	10.1 N	12.2 N
u réel (4N)	3.4 N	4.0 N	4.0 N
θ max	5°	15°	891°

À θ_0=18° : u demandé = 12.2 N >> U_max = 4 N → saturation permanente → θ atteint 891°

2. Commande LQR — Limites et Avantages

Avantages du LQR

Optimalité	Minimise rigoureusement $J = \int(X^TQX + uRu)dt$
Garantie théorique	$Re(\lambda_i) < 0$ garanti par la théorie
Stab. asymptotique	$\theta(t) \to 0$ et $x(t) \to 0$ prouvés mathématiquement
Simplicité	$u = -Kx$: une seule ligne de code
Calcul instantané	K calculé hors ligne, u en temps réel
Réglage intuitif	Q et R permettent d'ajuster performance / effort

Limites du LQR

Linéarisation	Valable uniquement pour $\theta \lesssim 15°$
Saturation	Sans U_max → force illimitée irréaliste
Divergence	$\theta_0 > 15°$ → saturation → divergence
Non-linéarités	Ne gère pas les grands écarts angulaires
Modèle requis	Nécessite les matrices A et B exactes
Sensibilité	Performances dégradées hors zone de linéar.

Ces limites motivent l'exploration de PPO : un agent qui apprend à stabiliser au-delà de $\theta \approx 15°$ sans modèle explicite.

3. Apprentissage par Renforcement — Principe

L'apprentissage par renforcement est la science de la prise de décisions séquentielles. Un agent autonome cherche à maximiser la somme des récompenses qu'il obtient en interagissant avec un environnement. À chaque itération, l'agent reçoit une observation et une récompense, puis choisit une action.

L'interaction entre un agent et son environnement est décrite figure 3.

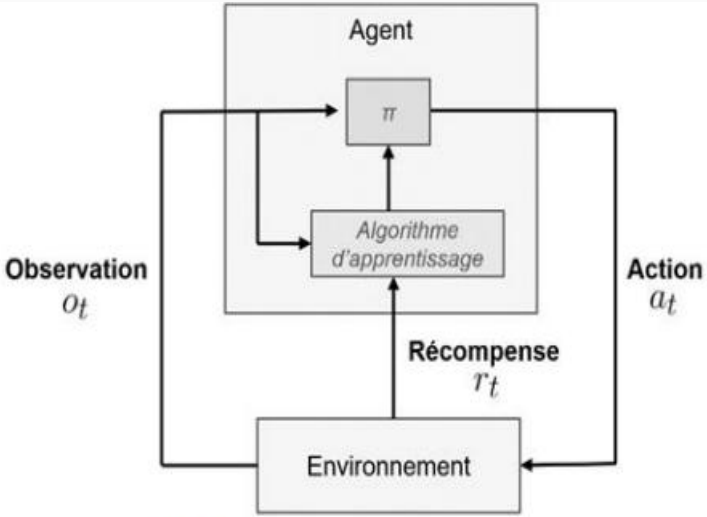


Figure 3 : Interaction entre l'agent et l'environnement

Simulation avec la méthode PPO (Proximal Policy Optimization)

Proximal Policy Optimization (PPO) est un algorithme d'apprentissage par renforcement profond (Deep Reinforcement Learning) développé par Open AI en 2017.

L'objectif fondamental de PPO est d'optimiser une politique (policy) tout en prévenant des mises à jour trop brutales qui pourraient déstabiliser le processus d'apprentissage.

PPO repose sur un principe simple mais puissant : effectuer des mises à jour de la politique par petites étapes contrôlées plutôt que par changements radicaux. Cette approche peut être résumée par l'adage suivant :
« Mieux vaut faire 100 petits pas sûrs qu'un grand pas risqué »

3.1.3 Architecture du modèle

PPO utilise deux réseaux de neurones distincts dont les rôles sont détaillés dans le tableau suivant :

Réseau	**Rôle**	**Entrée**	**Sortie**
Acteur (Actor)	Décision de l'action à prendre	État du système s_t	Probabilités des actions $\pi_\theta(a\|s)$
Critique (Critic)	Évaluation de la qualité de l'état	État du système s_t	Valeur estimée $V_\phi(s_t)$

TABLE 3.1 – Architecture acteur-critique de PPO

3.1.4 Mécanisme d'apprentissage

Collecte d'expériences

L'agent interagit avec l'environnement et stocke dans un buffer les transitions suivantes :

$$(s_t, a_t, r_t, \log \pi_{\text{ancien}}(a_t|s_t), \text{done}_t) \tag{3.1}$$

Calcul des avantages (Advantages)

PPO évalue chaque action en calculant son avantage, qui mesure si l'action était meilleure ou moins bonne que la moyenne :

$$A_t = \text{Avantage} = R_t - V_\phi(s_t) \tag{3.2}$$

où R_t est le retour actualisé et $V_\phi(s_t)$ la valeur prédite par le critique.

Pour réduire la variance, on utilise généralement l'estimation GAE (*Generalized Advantage Estimation*) :

$$A_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{3.3}$$

avec $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$

Le mécanisme de clipping — Innovation clé de PPO

PPO introduit un mécanisme de **clipping** qui limite l'amplitude des changements de politique. On définit le ratio de probabilité :

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ancien}}}(a_t|s_t)} \tag{3.4}$$

Ce ratio compare la nouvelle politique à l'ancienne :

— $r_t(\theta) > 1 \Rightarrow$ l'action est devenue plus probable

— $r_t(\theta) < 1 \Rightarrow$ l'action est devenue moins probable

Le clipping contraint ce ratio dans l'intervalle $[1-\epsilon, 1+\epsilon]$ avec $\epsilon = 0,2$ typiquement.

Fonction objectif

PPO optimise simultanément plusieurs objectifs à travers la fonction de perte vante :

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - c, 1 + c) A_t \right) \right]$$

La fonction objectif complète intègre également une pénalité d'entropie pour en rager l'exploration et une perte pour le critique :

$$L_t^{\text{totale}}(\theta, \phi) = \mathbb{E}_t \left[L_t^{\text{PPO}}(\theta) - c_1 L_t^{\text{value}}(\phi) + c_2 S[\pi_\theta](s_t) \right]$$

où :

- $L_t^{\text{value}}(\phi) = (V_\phi(s_t) - R_t)^2$ est la perte du critique
- $S[\pi_\theta](s_t)$ est le bonus d'entropie pour l'exploration
- c_1, c_2 sont des coefficients de pondération

3.1.5 Environnement personnalisé : PenduleInverseEnv

Définition

`PenduleInverseEnv` est un environnement personnalisé Gymnasium modélisant un **pendule inversé monté sur un chariot mobile**, avec des équations dynamiques non-linéaires identiques à celles utilisées par le contrôleur LQR de référence.

Caractéristiques principales

Élément	Description
Type	Pendule inversé sur chariot (cart-pole)
Objectif	Stabiliser $\theta = 0$ (verticale) **ET** $x = 0$ (centré)
État	$[x,\ \dot{x},\ \theta,\ \dot{\theta}]$ (4D)
Action	Force $u \in [-4,\ 4]$ N
Dynamique	Équations non-linéaires exactes (identique LQR)

Paramètres physiques (identiques au LQR)

Paramètre	Valeur	Signification
M	1.0 kg	Masse du chariot
m	0.1 kg	Masse du pendule
l	0.5 m	Longueur du pendule
g	9.81 m/s²	Gravité
dt	0.02 s	Pas de temps (50 Hz)

3.1.6 Présentation des fonctions clés du code PPO

- `_reward(self, state, u)`
  Entrées :
  — `state` : état courant
  — `u` : action
  Rôle : calcule la récompense pour guider l'apprentissage (stabilité, faible effort, angle réduit).

- `model.learn(total_timesteps, callback)`
  Entrées :
  — `total_timesteps` : nombre d'itérations
  — `callback` : suivi des performances
  Rôle : entraîne l'agent en interagissant avec l'environnement.

3.1.6 Présentation des fonctions clés du code PPO

- `reset(self, seed=None, options=None)`

  Entrées : paramètres optionnels

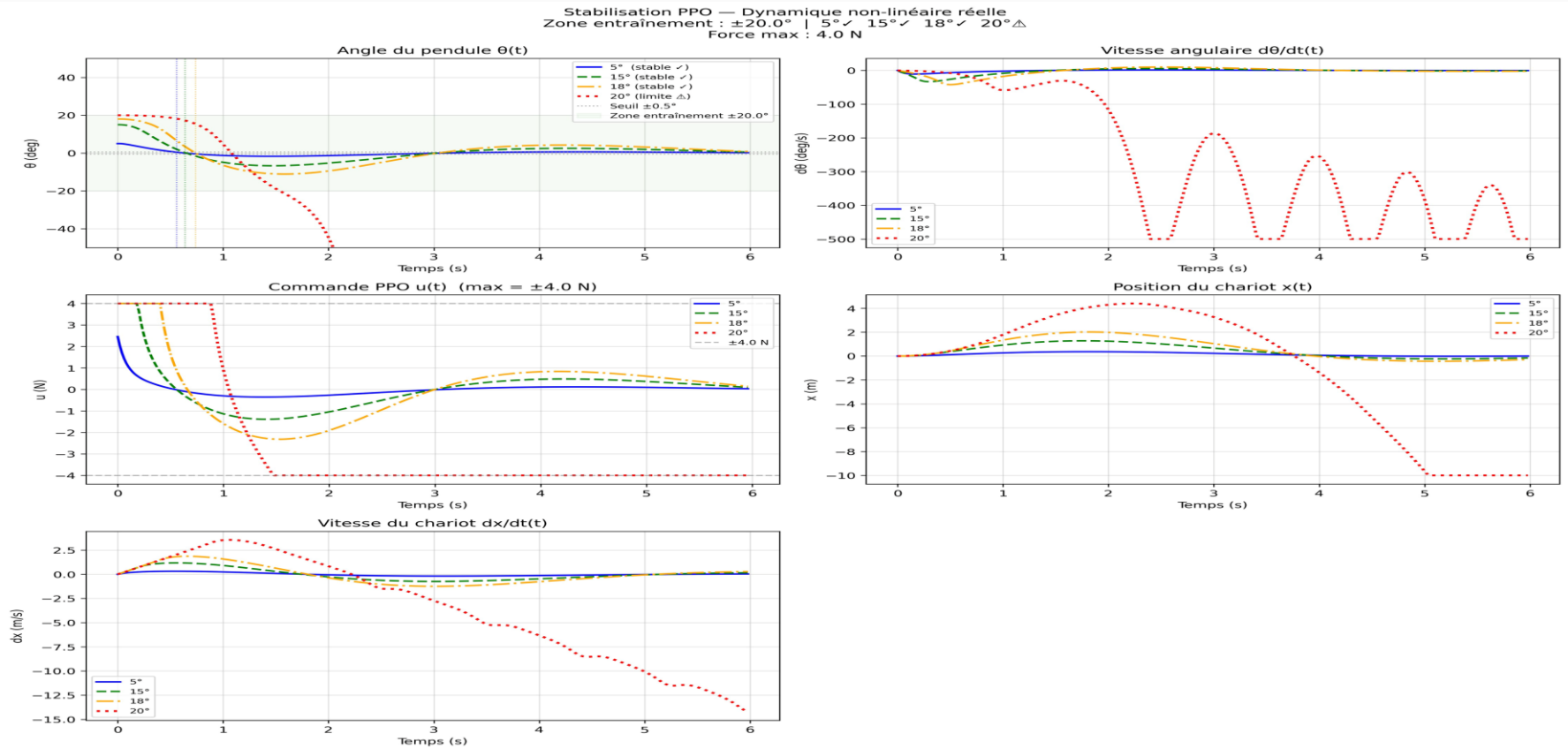
  Rôle : réinitialise l'environnement avec un état initial aléatoire.

- `step(self, action)`

  Entrée : `action`

  Rôle : met à jour l'état, calcule la récompense et détermine si l'épisode est terminé.

3. Résultats PPO — Stabilisation (U_max = 4 N, Zone entraîn. ±20°)



4. Comparaison — LQR vs PPO

Récapitulatif PPO vs LQR

Métrique	5°	15°	18°	20°
PPO — stab. (s)	0.56	0.64	0.74	1.08
LQR — stab. (s)	0.31	0.37	Diverge ✗	Diverge ✗
u max PPO (N)	2.5	4.0	4.0	4.0
Angle max (°)	5.0	15.0	18.0	1628.3
Stabilisation	△ Oscille	△ Oscille	△ Oscille	□ Diverge
Dans zone entraîn.	✓ oui	✓ oui	✓ oui	✓ oui


THANK YOU FOR
your attention